

The Code Breaker Documentation

Antony Gibson – ang80@aber.ac.uk

CS12320

Date: 30/04/2026

Contents

Contents Page	Page 1
Introduction	Page 2
Design	Page 5
Testing	Page 10
Evaluation	Page 34
Appendix	Page 35

Introduction

For this assignment, I have created a program that encrypts user input from a text file with a choice of three different cipher algorithms. Furthermore, there is functionality to decrypt ciphertext files using a choice of the same three cipher algorithms. This documentation describes the progression towards a final outcome of the project. I have implemented all of the required parts of this assignment, however I have not attempted any flair marks.

Use Case Diagram:

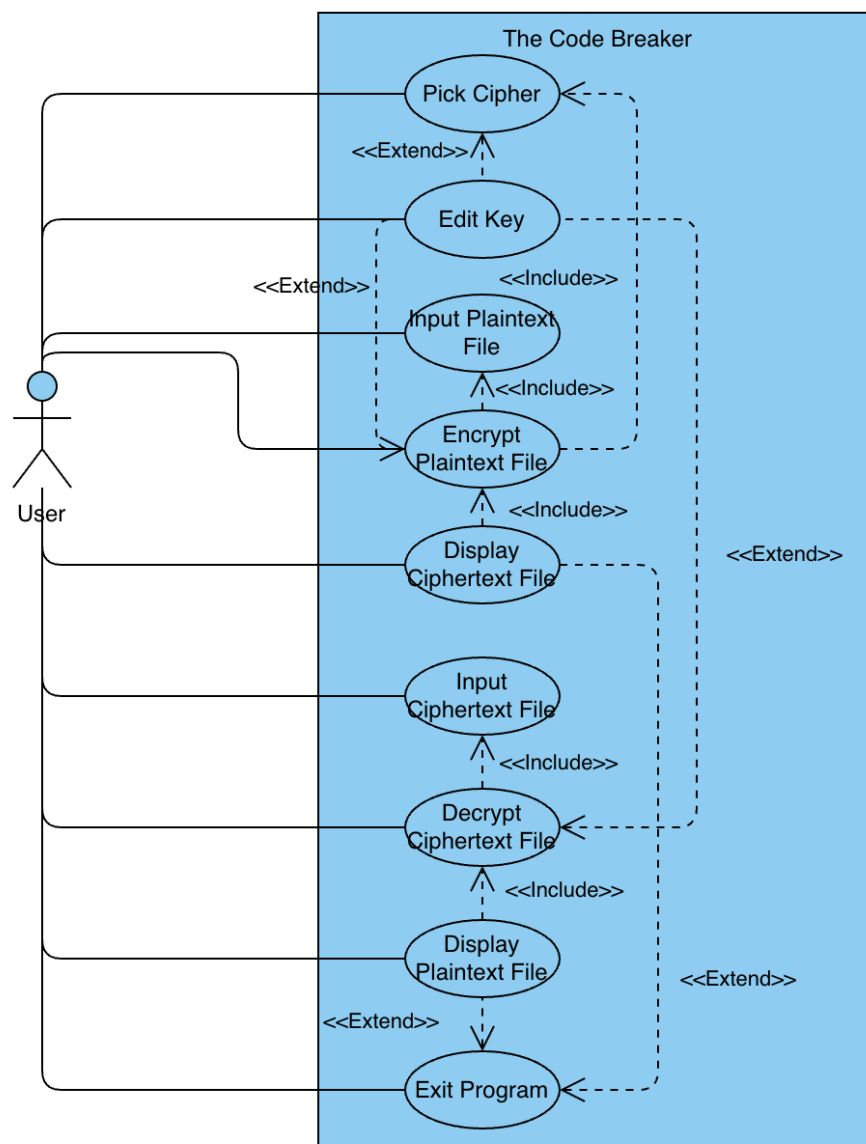


Figure 1 shows my final iteration of Use Case diagram.

Use Case Table:

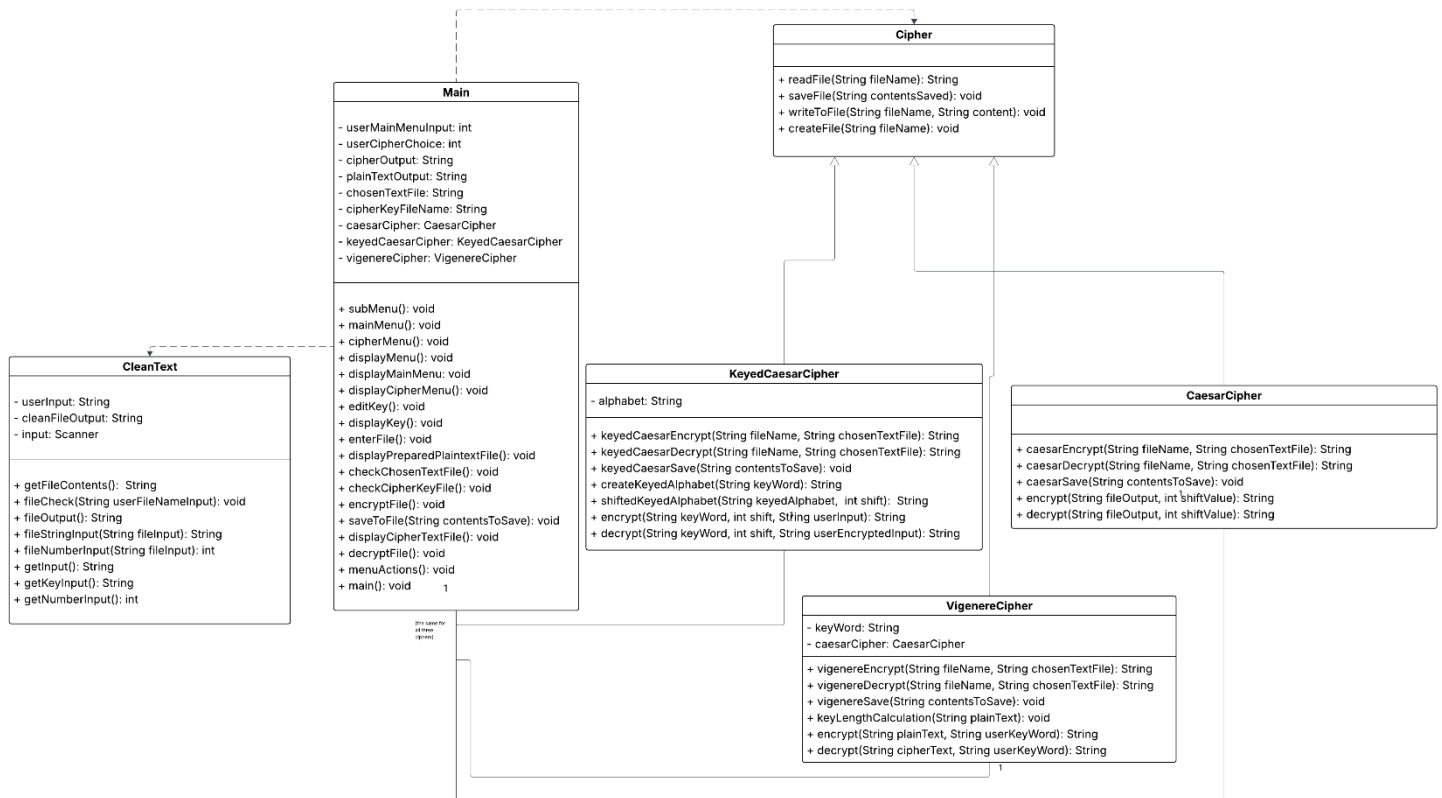
Use Case	Actor (who does it)	Outcome/Goal
Pick Cipher	User	User chooses which of the 3 available ciphers is used to encrypt the plaintext that they have inputted.
Edit Key	User	User changes the key that is used as a part of the cipher to encrypt their plaintext.
Input Plaintext File	User	User inputs a plaintext file into the program to be encrypted.
Input Ciphertext File	User	User inputs an already-encrypted ciphertext file into the program to be decrypted.
Encrypt Plaintext File	User	Previously inputted plaintext file is encrypted using the User's chosen cipher and key.
Decrypt Ciphertext File	User	Previously inputted ciphertext file is decrypted by the program back into plaintext.
Display Ciphertext File	User	Display the ciphertext of the inputted plaintext file to the user.
Display Plaintext File	User	Display the plaintext of the inputted ciphertext file to the user.
Exit Program	User	The user chooses to end the program, multiple exit codes used to determine whether exit was an error or user choice.

Base Use Case Table:

Base Use Case	Relationship	Other Use Cases	Condition (if <<extend>>)
Encrypt Plaintext File	<<include>>	Pick Cipher	-
Encrypt Plaintext File	<<include>>	Input Plaintext File	-
Display Ciphertext File	<<include>>	Encrypt Plaintext File	-
Decrypt Ciphertext File	<<include>>	Input Ciphertext File	-
Display Plaintext File	<<include>>	Decrypt Ciphertext File	-
Pick Cipher	<<extend>>	Edit Key	User chooses to modify key
Encrypt Plaintext File	<<extend>>	Edit Key	User edits key before encryption
Decrypt Ciphertext File	<<extend>>	Edit Key	User edits key before decryption
Exit Program	<<extend>>	Display Ciphertext File	User can view ciphertext before exiting
Exit Program	<<extend>>	Display Plaintext File	User can view plaintext before exiting

Design

Class Diagram:



The above photo shows my final iteration of class diagram. **A PDF file called ‘Class Diagram’ is available within the ZIP folder for this submission if this screenshot does not show the diagram in satisfactory quality.**

Also relevant to the design section is the program exit codes that I have elected to use:

- Exit Code 0 is used when the user has chosen to quit the program.
- Exit Code 1 is used when the program has an error or has an input it cannot handle.

I have used these values as it provides more consistency and clarity as to what went wrong in the event the program exits, and also means a less confusing error message for the user.

Class Descriptions:

Main

This is, somewhat unsurprisingly, the main class of the program. This handles calling all menus and carrying out the corresponding menu functions. All other classes are called from Main at various points during program execution, which means that by definition Main has a one-to-one relationship with all three cipher subclasses, and dependency relationships with CleanText and the Cipher superclass. This is because it calls methods from within both classes at various points.

- subMenu(), mainMenu() and cipherMenu() – contain the print statements for the menus that are outputted to the user.
- displayMenu() – checks if user wants to bypass initial submenu, which forces picking a cipher before continuing, to see the entire main menu. A switch case statement is used to map user input into the desired outputs, in this case which menu to display to the user.
- displayMainMenu() - calls mainMenu() to display it, and then takes user input as user is choosing menu items, and passes these inputs into menuActions().
- displayCipherMenu() - uses a switch case statement to map user's choice of cipher to the corresponding key file and maps the user's input choice to the corresponding cipher. This is then stored in a global variable and is accessed by most other methods, for example editKey() or encryptFile().
- editKey() – takes user input for a new key value and writes it to the corresponding file defined in displayCipherMenu(). Uses a switch case statement to change the output to the console to help guide users' inputs to give the correct values depending on the type of cipher chosen.
- displayKey() – displays the contents of the key file of the chosen cipher. Checks if the cipherKeyFileName variable is null before outputting the key value: this is a check to ensure the user has a cipher key file chosen.
- enterFile() – prompts user to enter a filename, using a function of the CleanText class. Used in multiple methods, whenever a file is needed to be inputted into the program.
- displayPreparedPlaintextFile() – calls methods from both Cipher and CleanText classes, to write the prepared plain text to the prep.txt file and then output this to the user. Checks if chosenTextFile variable is null before printing prepared plain text: ensures user has inputted a file before trying to prepare plain text.
- checkChosenTextFile() and checkCipherKeyFile() – check chosenTextFile and cipherKeyFile variables respectively aren't null, used to ensure null values not being passed into other methods. Used in encryptFile() and decryptFile().
- encryptFile() and decryptFile() – switch case statements used based on the choice of cipher made by the user. Calls the [cipher name]Encrypt and [cipher

name]Decrypt methods, which in turn call the actual encrypt or decrypt methods of the cipher.

- `saveToFile()` – checks if user has chosen a cipher, and whether there is an encrypted file that needs saving. Uses a switch case statement to hand contents to save to file to the [cipher name]Save methods.
- `displayCipherTextFile()` – checks if `cipherOutput` variable has been used, to ensure user has encrypted a file, before displaying the resulting ciphertext to the user.
- `menuActions()` – a switch case statement that calls most of the methods outlined above depending on user input. Used in conjunction with `displayMainMenu()` to essentially create a loop to ensure a menu is always displayed. `menuActions()` is what maps user's number inputs from `mainMenu()` onto the corresponding actions.
- `main()` – calls `displayMenu()` and starts the program.

CleanText

This is the class I use to sanitise inputs, for both console and file inputs. This contains only static methods and variables because it is a class of utility features. Main has a dependency relationship with CleanText.

- `getFileContents()` – checks a file exists with the name given by the user, then reads the file and cleans up file output using `fileStringInput()`. Uses an if statement to ensure that the output from the file is never an empty string.
- `fileCheck()` – used to check if file chosen by user is a text file, exists and can be read by the program. If the file fails one or more of these checks then an error is thrown and the program exits.
- `fileOutput()` – returns the result of `getFileContents()` to other methods that haven't called `getFileContents()` themselves. This allows the file output to be used in methods without having to get the user to re-enter the file name and re-do all of the checks on the file.
- `fileStringInput()` – uses regex to get rid of whitespace and then removes any characters that aren't in the alphabet. This is used when a text output is required from a file.
- `fileNumberInput()` – same as `fileStringInput()`, but removes anything that isn't a number. Method will return -1 if the file it is checking has no numbers in, to prevent program crashing if method calling `fileNumberInput()` from elsewhere is expecting an int value back. This is used when a numerical output is required from a file.
- `getInput()` – uses Scanner to read user input and returns. This is used as the basis for all of the classes that take user input from the console.

- `getKeyInput()` – calls `getInput()`, then trims and capitalises the result. This is used, entirely unsurprisingly, for getting user input in the console to write to the key file.
- `getNumberInput()` – calls `getInput()`, then replaces anything that isn't a single digit integer. There is a try catch loop to catch number format exception errors. The `userNumberInput` variable is also initialised as -1 to ensure there is always a numerical value passed back to methods that are requiring an integer response from this method.

Cipher

This is the superclass extended by all of the cipher subclasses. Main has a dependency relationship with Cipher, and CaesarCipher, KeyedCaesarCipher and VigenereCipher all extend Cipher. Cipher contains all of the file handling methods that are used within my program, which are then used in each of the subclasses.

- `readFile()` – creates a `StringBuilder` object to append each line to, then reads the file line by line. Each line is trimmed and capitalise and then is appended to the output string. Also uses a try catch loop to catch IO Exception errors.
- `saveFile()` – writes any input passed into it to the chosen file using `writeToFile()`.
- `writeToFile()` – creates a `FileWriter` object, which is used to write all content passed into the method to the chosen file. This method also calls `createFile()` to ensure a file with the inputted name is created even if it didn't already exist. There is also a try catch loop to catch any IO Exception errors.
- `createFile()` – a new `File` object is created, and if a file with the inputted name doesn't exist then one is created. A try catch loop is used to catch any IO Exception errors. If there is already a file that exists with the inputted name, then no new file is created.

CaesarCipher

Subclass of Cipher that handles encryption and decryption of Caesar Cipher inputs. CaesarCipher is also used quite extensively in my implementation of the Vigenere Cipher.

- `caesarEncrypt()` – gets key value and calls the `encrypt` method to return result to `Main()`.
- `caesarDecrypt()` – same as above, used to decrypt and return to `Main()`.
- `caesarSave()` – saves anything passed into the method to the specified file.
- `encrypt()` – shifts the user input by the shift value specified in the `caesar-key.txt` file.
- `decrypt()` – calls the `encrypt` function with the same shift value (except negative) to return to plaintext.

KeyedCaesarCipher

Subclass of Cipher that handles encryption and decryption of Keyed Caesar Cipher inputs.

- keyedCaesarEncrypt() – gets shift value and key word, and passes into encrypt function. Returns result to Main().
- keyedCaesarDecrypt() – same as above, used to decrypt and return value to Main().
- keyedCaesarSave() – saves anything passed into the method to the specified file.
- createKeyedAlphabet() – creates new alphabet in the format required for keyed Caesar, with the key word appended to the beginning of the normal alphabet.
- shiftedKeyedAlphabet() – shifts the alphabet by the shift value specified by the user
- encrypt() – finds position in normal alphabet of each plaintext character inputted, then outputs the character in the same position in the shifted keyed alphabet.
- decrypt() – re-constructs the shifted keyed alphabet, then finds the position in the new alphabet of the cipher text inputted. Outputs the letter in the same position in the normal alphabet.

VigenereCipher

Subclass of Cipher that handles encryption and decryption of Vigenere Cipher inputs. Uses Caesar Cipher in the encryption and decryption methods.

- vigenereEncrypt() – takes key word and calls encrypt function, returns result to Main().
- vigenereDecrypt() – same as above, calls decrypt and returns result to Main().
- vigenereSave() – saves anything passed into the method to the specified file.
- keyLengthCalculation() – finds difference in length between key word and the plaintext, then either extends or reduces length of the key to match the plaintext length exactly.
- encrypt() – finds position of each character of the key, then calls a Caesar cipher on each character of plaintext with the shift value of the position of the key word character.
- decrypt() – passes ciphertext and the same shift value into the Caesar cipher decrypt method, which reverts back to plaintext.

Testing

Test Table:

ID	Requirement	Description	Inputs	Expected Outputs	Pass / Fail	Comments
1.1	FR1	Choose between the 3 ciphers	1, then 1	Caesar Cipher	Pass	
1.2			1, then 2	Keyed Caesar Cipher	Pass	
1.3			1, then 3	Vigenere Cipher	Pass	
1.4			1, then 4	Display main menu	Pass	
1.5			1, then 5	Error message to pick valid option	Pass	
1.6			1, then 0	Exit program with exit code 0	Pass	
1.7			1, then Antony	Error message to prompt picking a number	Pass	
2.1	FR2	Edit the key	2, then 2	Prompt to enter Cipher	Pass	Cipher hasn't been chosen
2.2			1, then 1 2 or 3, then 2	Take input to store in key file	Pass	Cipher has been chosen
3.1	FR3	Display the key	1, then 1 2 or 3, then 3	Key to be displayed	Pass	cipher has already been chosen
3.2			2, then 3	Prompt to pick cipher	Pass	cipher hasn't already been chosen

4.1	FR4	Input a user-specified, plaintext file	1 or 2, then 4	Prompt to input file name	Pass	
4.2			File name that already exists	File to be read by program	Pass	
4.3			File name that doesn't already exist	Error message thrown	Pass	
4.4			File name that isn't followed by .txt	Error message to be thrown	Pass	
5.1	FR5	Display the user file as prepared plain text	1 or 2, then	File to be displayed as prepared plaintext	Pass	Plaintext file has already been inputted
5.2				Error message to prompt inputting a plaintext file first	Pass	Plaintext file hasn't already been inputted
6.1	FR6	Encrypt the prepared plaintext using the chosen cipher	1, then 1 2 or 3, then 6	Prompt to enter file name to encrypt	Pass	Cipher has already been chosen
6.2			2, then 6	Prompt to choose cipher	Pass	Cipher hasn't already been chosen
6.3			1, then 1 2 or 3, then 4 (with a file name), then 6	Nothing	Pass	Once 6 is chosen, just encrypts with no further input
7.1	FR7	Display ciphertext	1, then 1 2 or 3, then 4, then 6, then 7	Output of file encryption to be shown	Pass	Encryption has already occurred

7.2			2, then 7	Prompt to encrypt first	Pass	Cipher hasn't been chosen
7.3			1, then 1 2 or 3, then 4, then 7	Prompt to encrypt file first	Pass	Encryption hasn't already occurred
8.1	FR8	Save ciphertext to a user-specified file	1, then 1 2 or 3, then 4, then 6, then 8	Prompt to enter file name to save to	Pass	Encryption has already occurred
8.2			2, then 8	Prompt to encrypt file first	Pass	Cipher hasn't been chosen
8.3			1, then 1 2 or 3, then 4, then 8	Prompt to encrypt a file first	Pass	Encryption hasn't already occurred
8.4			File name that already exists	Save to file	Pass	
8.5			File name that doesn't already exist	Create file then save to it	Pass	
8.6			File name that doesn't end in .txt	Create file anyway	Pass	
9.1	FR9	Input a ciphertext file	1 or 2, then 9	Prompt to enter filename	Pass	
9.2			Enter a file that exists	File to be read	Pass	
9.3			Enter a file that doesn't exist	Error message to be thrown	Pass	
9.4			Enter a file that doesn't end in .txt	Error message to be thrown	Pass	

10.1	FR10	Decrypt ciphertext to prepared plaintext	1, then 1 2 or 3, then 9, then 10	Decrypt file	Pass	Assuming cipher has already been chosen
10.2			2, then 9, then 10	Prompt to choose cipher	Pass	Assuming cipher hasn't already been chosen
10.3			1, then 1 2 or 3, then 10	Prompt to input a file first	Pass	Once file chosen, encrypts
10.4			2, then 10	Prompt to choose a cipher first	Pass	
11.1	FR11	Exit the program	0	Exit the program with exit code 0	Pass	
12.1	NFR1	Program must have a command line menu	Antony	Error to prompt number input	Pass	
12.2			:)	Error to prompt number input	Pass	

Screenshots from Testing

1.1

```

1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
1
1. Caesar Cipher
2. Keyed Caesar Cipher
3. Vigenere Cipher
4. Display Other Menu Options
0. Exit Program
1

Cipher Selected. Now pick what you're going to do with it.
```

1.2

```
1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
1
1. Caesar Cipher
2. Keyed Caesar Cipher
3. Vigenere Cipher
4. Display Other Menu Options
0. Exit Program
2
Cipher Selected. Now pick what you're going to do with it.
```

1.3

```
1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
1
1. Caesar Cipher
2. Keyed Caesar Cipher
3. Vigenere Cipher
4. Display Other Menu Options
0. Exit Program
3
Cipher Selected. Now pick what you're going to do with it.
```

1.4

```
1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
1
1. Caesar Cipher
2. Keyed Caesar Cipher
3. Vigenere Cipher
4. Display Other Menu Options
0. Exit Program
4
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
```

1.5

```
1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
1
1. Caesar Cipher
2. Keyed Caesar Cipher
3. Vigenere Cipher
4. Display Other Menu Options
0. Exit Program
5
Please choose one of the available options. Try again.
1. Caesar Cipher
2. Keyed Caesar Cipher
3. Vigenere Cipher
4. Display Other Menu Options
0. Exit Program
```

1.6

```
1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
1
1. Caesar Cipher
2. Keyed Caesar Cipher
3. Vigenere Cipher
4. Display Other Menu Options
0. Exit Program
0
Process finished with exit code 0
```

1.7

```
1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
1
1. Caesar Cipher
2. Keyed Caesar Cipher
3. Vigenere Cipher
4. Display Other Menu Options
0. Exit Program
Antony
Please choose one of the available options. Try again.
1. Caesar Cipher
2. Keyed Caesar Cipher
3. Vigenere Cipher
4. Display Other Menu Options
0. Exit Program
```

2.1

```
1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
2
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
2
Please enter a cipher before continuing.
```

2.2

```
1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
1
1. Caesar Cipher
2. Keyed Caesar Cipher
3. Vigenere Cipher
4. Display Other Menu Options
0. Exit Program
2

Cipher Selected. Now pick what you're going to do with it.
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
2
Please enter the word to use as a key, followed by the shift value you would like.
|
```

3.1

```
1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
1
1. Caesar Cipher
2. Keyed Caesar Cipher
3. Vigenere Cipher
4. Display Other Menu Options
0. Exit Program
1

Cipher Selected. Now pick what you're going to do with it.
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
3
Here is the current key:
3
```

3.2

```
1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
2
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
3
Please select a cipher before continuing.
```

4.1

```
1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
2
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
4
Please enter the file name:
```

4.2

```
1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
2
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
4
Please enter the file name:
testDoc.txt
To continue, select another item from the menu.
```

4.3

```
1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
2
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
4
Please enter the file name:
antonyIsAwesome.txt
An Error has occurred: antonyIsAwesome.txt (No such file or directory)
Please try again.
```

4.4

```
1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
2
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
4
Please enter the file name:
testDoc
File error. Program could not access file, please try again.
```

5.1

```
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
4
Please enter the file name:
testDoc.txt
To continue, select another item from the menu.
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
5
Saved to File.
Prepared Plaintext File: THISISATEST
```

5.2

```
1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
2
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
5
Please enter a plaintext file before continuing.
```

6.1

```
1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
1
1. Caesar Cipher
2. Keyed Caesar Cipher
3. Vigenere Cipher
4. Display Other Menu Options
0. Exit Program
2

Cipher Selected. Now pick what you're going to do with it.
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
6
Please enter a file to encrypt or decrypt.
```

6.2

```
1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
2
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
6
Please enter a cipher before trying to encrypt a file.
```

6.3

```
1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
1
1. Caesar Cipher
2. Keyed Caesar Cipher
3. Vigenere Cipher
4. Display Other Menu Options
0. Exit Program
2
Cipher Selected. Now pick what you're going to do with it.
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
4
Please enter the file name:
testDoc.txt
To continue, select another item from the menu.
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
6
```

7.1

```

1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
1
1. Caesar Cipher
2. Keyed Caesar Cipher
3. Vigenere Cipher
4. Display Other Menu Options
0. Exit Program
1

Cipher Selected. Now pick what you're going to do with it.
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
4
Please enter the file name:
testDoc.txt
To continue, select another item from the menu.
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
6

```

```

1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
7
WKLVLVDWHVW

```

7.2

```

1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
2
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
7
Please encrypt a file before trying to display it.

```

7.3

```

1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
1
1. Caesar Cipher
2. Keyed Caesar Cipher
3. Vigenere Cipher
4. Display Other Menu Options
0. Exit Program
1

Cipher Selected. Now pick what you're going to do with it.
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
4

Please enter the file name:
testDoc.txt
To continue, select another item from the menu.
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
7
Please encrypt a file before trying to display it.

```

8.1

```

1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
1
1. Caesar Cipher
2. Keyed Caesar Cipher
3. Vigenere Cipher
4. Display Other Menu Options
0. Exit Program
1

Cipher Selected. Now pick what you're going to do with it.
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
4

Please enter the file name:
testDoc.txt
To continue, select another item from the menu.
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
6

```

```

1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
8

***This will overwrite any existing file contents***
Press 1 to save, press 0 to go back.
1
Please enter the filename to save to:

```

8.2

```
1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
2
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
8
Please encrypt or decrypt a file before trying to save.
```

8.3

```
1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
1
1. Caesar Cipher
2. Keyed Caesar Cipher
3. Vigenere Cipher
4. Display Other Menu Options
0. Exit Program
1
Cipher Selected. Now pick what you're going to do with it.
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
4
Please enter the file name:
testDoc.txt
To continue, select another item from the menu.
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
8
Please encrypt or decrypt a file before trying to save.
```

8.4

```
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
8
***This will overwrite any existing file contents***
Press 1 to save, press 0 to go back.
1
Please enter the filename to save to:
testDoc.txt
Saved to File.
```

8.5

```
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
8
***This will overwrite any existing file contents***
Press 1 to save, press 0 to go back.
1
Please enter the filename to save to:
antonyIsAwesome.txt
File created: antonyIsAwesome.txt
```

8.6

```
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
8
***This will overwrite any existing file contents***
Press 1 to save, press 0 to go back.
1
Please enter the filename to save to:
antonyIsAwesome
File created: antonyIsAwesome
```

9.1

```
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
9
Please enter the file name:
```

9.2

```
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
9
Please enter the file name:
testDoc.txt
To continue, select another item from the menu.
```

9.3

```
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
9
Please enter the file name:
pinkFluffyUnicorns.txt
An Error has occurred: pinkFluffyUnicorns.txt (No such file or directory)
Please try again.

Process finished with exit code 1
```

9.4

```
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
9
Please enter the file name:
pinkFluffyUnicorns
File error. Program could not access file, please try again.

Process finished with exit code 1
|
```

10.1

```
9
Please enter the file name:
testDoc.txt
To continue, select another item from the menu.
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
10
Decrypted File: QEFPPXQBPQ
```

10.2

```
9
Please enter the file name:
testDoc.txt
To continue, select another item from the menu.
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
10
Please enter a cipher before trying to decrypt a file.
```

10.3

```
1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
1
1. Caesar Cipher
2. Keyed Caesar Cipher
3. Vigenere Cipher
4. Display Other Menu Options
0. Exit Program
1

Cipher Selected. Now pick what you're going to do with it.
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
10
Please enter the file name:
testDoc.txt
Please enter a file to encrypt or decrypt.
Please enter the file name:
testDoc.txt
To continue, select another item from the menu.
Decrypted File: QEFPPXQBPQ
```

10.4

```

1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
2
1. Pick your Cipher
2. Edit Key
3. Display Key
4. Input Plaintext File
5. Display Prepared Plaintext File
6. Encrypt File
7. Display Encrypted File
8. Save Encrypted File
9. Input Ciphertext File
10. Decrypt and Save Ciphertext File
0. Exit Program
10
Please enter a cipher before trying to decrypt a file.

```

11.1

```

1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
0

Process finished with exit code 0

```

12.1

```

1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
Antony
Please enter an available option.

```

12.2

```

1. Pick your Cipher
2. Display Other Menu Options
0. Exit Program
:)
Please enter an available option.

```

Testing Discussion

I have shown, to the best of my ability, my code when presented with as many tests as I could think of.

During this testing, I encountered no significant errors – any that I did find were easily fixable enough for me to fix during my testing. Therefore, I consider this to be a successful test of my code and I am confident that the array of tests I have performed will equate to relative stability when being used.

Evaluation

How I solved the Assignment:

I worked from basics upwards, leaving the Ciphers until last to ensure I had the relevant associated methods to make implementing them easier and ensure that they were designed to work with the rest of my code. For me, this made it easier to implement the ciphers and allowed me to get comfortable with java in the smaller methods before tackling the larger and more complex cipher methods.

I did not attempt any flair marks on this assignment.

What I found difficult:

Getting used to object oriented programming as opposed to the procedural programming I have used in the past. My first draft of diagrams illustrates this, as I was relatively clueless as to how any of the program would actually work until I started implementing code. The diagrams are pretty inaccurate in comparison to my final product.

What remains to be done?

All required functionality is implemented, aside from flair marks, therefore I consider this project to be entirely complete.

What have I learned?

How to use object-oriented programming! And the importance of proper planning before beginning to implement code. Using the knowledge I have gained from this project, if I were to start again, I would have much better initial plans.

What mark do I believe I should be given?

I believe that my code is of a satisfactory standard to achieve a mark in the region of 60/100, around 60%.

Appendix

Final Wordcount:

1,916

References:

<https://www.geeksforgeeks.org/java/stringbuilder-class-in-java-with-examples/> - helped with StringBuilder implementation

<https://www.geeksforgeeks.org/java/file-class-in-java/> - helped with different functions
File class can be used for

<https://www.geeksforgeeks.org/java/java-string-valueof/> - used for finding
String.valueOf() in Vigenere Cipher

https://www.w3schools.com/java/java_try_catch.asp - used for the different error
types in try catch statements

Gitlab Timepoint Progression:

Some uploads are from the day before the timepoint date. Where possible, I have used an upload from the stated date but in the cases where no upload exists for the date stated, I have used the closest prior upload to the timepoint date.

Timepoint 1: 13th March

For my initial development, I focussed on familiarising myself with taking inputs and changing them into prepared plain text. I did create classes for the ciphers and a menu class that I subsequently removed, but these remained empty in this upload.

My Main class in this upload had a method called getUserInput() which created a CleanText object and then called the getInput() method in CleanText to get user to enter the type of cipher they wanted to use.

My CleanText class had a Scanner object implemented, which I used to do .nextLine() to take user input. There was also some regex implementation, to remove whitespace and all non-letter characters. However, output from this was printed directly to console.

Timepoint 2: 20th March

For the second timepoint, I firstly removed the menu class because at the time I deemed it to be surplus to requirements. The classes implemented in upload 1 for Cipher, CaesarCipher, KeyedCaesarCipher and VigenereCipher all remained empty in this upload too, with my developments focussed around Main and CleanText.

My Main class now had separate methods for getting user input from console, input for ciphers and input for numbers. Furthermore, I implemented the mainMenu() and cipherMenu() methods used to print the menus to the console. These were already pretty much in their final form, with only wording changes made to them from this point onwards. I implemented a dedicated displayMenu() method to handle displaying the menu and taking user input, and also linking the user input to the new menuActions() method. menuActions() is the method I use to take user input and then perform the corresponding operations to each menu item. Also, a separate displayCipherMenu() method appeared to do in one method what the combo of the displayMenu() and menuActions() methods did for the main menu.

The CleanText class also had more changes, with the dedicated getInput(), getCipherInput() and getNumberInput() methods. getInput() was called by both CipherInput() and NumberInput(), which then used regex to remove numbers and letters respectively.

Timepoint 3: 27th March

For the third timepoint CaesarCipher, KeyedCaesarCipher and VigenereCipher remained empty, however I developed the Cipher class as well as Main. CleanText remained unchanged in this upload.

My Main class contained minor tweaks to the already-uploaded code, such as now associating the user input on the cipher menu with a cipher. I also migrated from an if else statement in menuActions() to a switch case statement. This was accompanied by a fleshing out of the available options, with implementations for editing the key file, by associating each of the key file names with the number the user had inputted when choosing a cipher, and displaying the contents of the key file using the same logic. The editing and displaying of these file contents was allowed by my additions to Cipher, of the readFile(), writeToFile() and createFile() methods.

The Cipher class changes added the methods outlined above, with their functionality being relatively self-explanatory:

- readFile() was used for reading a file, by taking the file name and reading it line-by-line, before outputting each line to console. It also used try catch statements to check for IOException errors.

- `writeToFile()` was used for modifying file contents, by creating a `FileWriter` object and calling `.write()` on the file name passed into the method. It also used try catch statements to check for `IOException` errors.
- `createFile()` was used to create a file if one with the name passed into `writeToFile()` didn't already exist. This was achieved by calling a `File` object. It also used try catch statements to check for `IOException` errors.

Timepoint 4: 24th April

For the fourth timepoint all classes except `VigenereCipher` had been developed. It was a productive easter break!

The Main class now called `CaesarCipher` and `KeyedCaesarCipher` objects, and had a plethora of new methods, such as:

- `subMenu()`: first menu seen by user, to try and force picking a cipher before continuing
- `enterFile()`: called `CleanText` `getFileContents()` function, which gets user to enter a file name.
- `displayPreparedPlaintextFile()`: takes file input, makes it prepared plain text, writes to `prep.txt` file as specified in the brief, and then outputs contents of `prep.txt` to the user.
- `encryptFile()`: uses switch case statement to associate the correct cipher encrypt based on the number the user entered when picking a cipher. No encrypt functionality for `vigenere` in this upload because I had yet to implement it.
- `saveFile()`: gets filename to save to, checks with user if they want to save to the file and then writes to file.
- `displayCipherTextFile()`: prints the cipher text output from encrypting the file to the user
- `decryptFile()`: uses almost identical logic to the `encryptFile()` method.

Furthermore, I made updates to `menuActions()` to ensure all choices now linked to relevant methods. `displayCipherMenu()` also got updates, a switch case statement assigning the key file name at the point of choosing the cipher. Further, the unnecessary calling of methods in the `main()` method at the bottom.

The `CleanText` class also got a makeover, making the methods static because they are utility files and so being static makes more sense than creating multiple instances of `CleanText`. More new methods have also been added, such as:

- `getFileContents()` and `fileCheck()`: `fileCheck` checks the file exists, can be read and is a text file, and then `getFileContents` reads the file and uses another new

method called `fileStringInput()` to remove whitespace and leave only letter characters in the file input.

- `fileNumberInput()` performs much the same function as `fileStringInput()`, but with numbers instead.

There were few changes to Cipher, aside from removing the ability for `readFile` to create a new file (which was an error in my original implementation) and appending each line to a variable to return instead of printing to console.

`CaesarCipher` gained an `encrypt()` and `decrypt()` method, with `encrypt()` finding position of each character in the normal alphabet then applying the shift value to get the new position in the alphabet for each character. `decrypt()` works by calling `encrypt` but with a negative shift value, to revert back to plain text.

`KeyedCaesarCipher` was also fully implemented, by creating a keyed alphabet then shifting the alphabet before encrypting the plaintext with the new alphabet. Creating the keyed alphabet was done by removing duplicate characters from the key word, then appending any remaining letters of the alphabet. Shifting the alphabet was done by using mostly the same logic as used to shift the Caesar cipher. `encrypt()` was done by finding the position of each plain text character in the normal alphabet then returning the character at the same position in the shifted keyed alphabet. `decrypt()` was implemented by reconstructing the shifted and keyed alphabet, then finding the position of the ciphertext character in that alphabet and returning the character at the same position in the normal alphabet.

Timepoint 5: 1st May

For the final timepoint, all classes have been developed.

The Main class now calls objects for all 3 ciphers, and has gained some new methods such as:

- `checkChosenTextFile()`: checks the user has inputted a text file
- `checkCipherKeyFile()`: checks the cipher key file to ensure the user has entered a key
- `saveToFile`: replacing `saveFile`, checks user has entered a cipher and has the output from a cipher operation ready to save. Then hands the output to the `save` method in the chosen cipher class to save to a file chosen by the user.

Other methods have had updates, such as:

- `editKey()` now requires entering a cipher choice before attempting to edit a key file and has different prompts for each cipher so the user knows what

information to input into the key file. It also checks that the user has entered something in the key file before saving.

- displayKey() now checks that a cipher has been chosen before attempting to find a key file
- displayPreparedPlaintextFile() now prompts user to enter a text file if one hasn't already been entered.
- encryptFile() and decryptFile() now prompt user to pick a cipher before encrypting/decrypting if no cipher is selected. encryptFile() runs checks on the chosenTextFile and cipherKeyFile variables before handing them off to the methods in the chosen cipher class to encrypt, while decryptFile() checks that a file has been entered to be decrypted before handing off to methods in the chosen cipher class to decrypt.
- displayCipherTextFile() now checks user has encrypted a file before attempting to display ciphertext.

The only change to CleanText was removal of the redundant getCipherInput() method.

The Cipher superclass remained unchanged aside from the addition of the saveFile() method from Main().

CaesarCipher gained a caesarEncrypt(), caesarDecrypt() and caesarSave() set of functions. These utilise the file handling methods in the Cipher superclass and replace the previous method of calling encrypt directly from Main().

KeyedCaesarCipher gained the same trio of methods, keyedCaesarEncrypt(), keyedCaesarDecrypt() and keyedCaesarSave(). Once again, these are using the file handling methods in the Cipher superclass and replace the previous code calling the encrypt function directly from Main().

VigenereCipher has been implemented, with the addition of the trio of file handling classes, vigenereEncrypt(), vigenereDecrypt() and vigenereSave(), as well as:

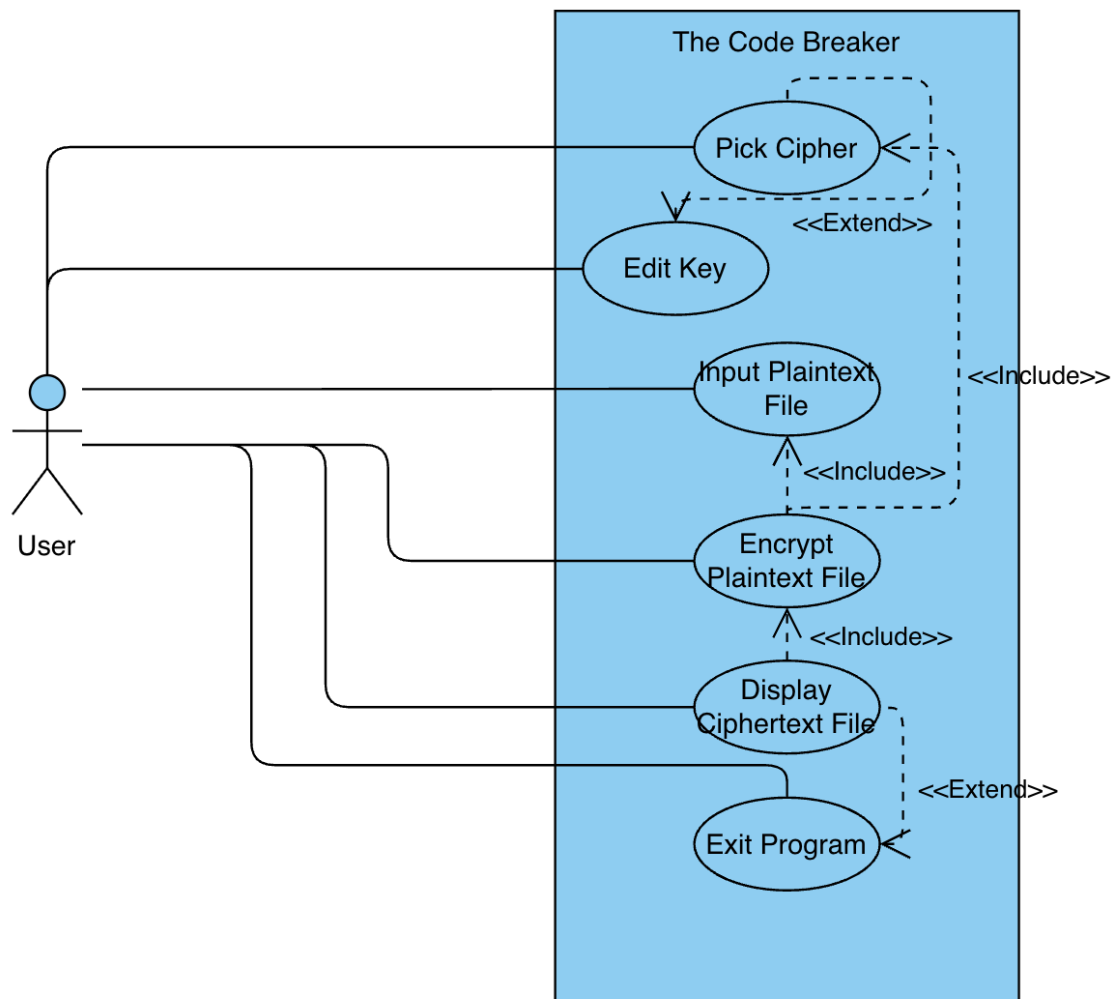
- keyLengthCalculation(): this makes sure the key word used in the cipher is the exact length of the plaintext, by either repeating the key word until length is matched or by removing letters.
- encrypt() : encrypts the plaintext by calling a Caesar cipher on the first letter of plaintext with a shift of the ASCII value of the first letter of the key word. This repeats until entire plaintext input is encrypted. It is then returned.
- decrypt(): works the same in reverse, determines the shift applied by finding the ASCII value of the first ciphertext letter and then using a negative shift by that value to give the plaintext letter. Repeat until all of the ciphertext is decrypted. This is then returned.

All classes and methods in this final timepoint now have Javadoc comments.

Use Case Tables and Diagrams:**Version 1:**

Use Case	Actor (who does it)	Outcome/Goal
Pick Cipher	User	User chooses which of the 3 available ciphers is used to encrypt the plaintext that they have inputted.
Edit Key	User	User changes the key that is used as a part of the cipher to encrypt their plaintext.
Input Plaintext File	User	User inputs a plaintext file into the program to be encrypted.
Encrypt Plaintext File	User	Previously inputted plaintext file is encrypted using the User's chosen cipher and key.
Display Ciphertext File	User	Display the ciphertext of the inputted plaintext file to the user.
Exit Program	User	The user chooses to end the program, multiple exit codes used to determine whether exit was an error or user choice.

Base Use Case	Relationship	Other Use Case	Condition (if <<extend>>)
Encrypt Plaintext File	<<include>>	Pick Cipher	-
Encrypt Plaintext File	<<include>>	Input Plaintext File	-
Display Ciphertext File	<<include>>	Encrypt Plaintext File	-
Edit Key	<<extend>>	Pick Cipher	Only occurs if user chooses to modify the key.
Exit Program	<<extend>>	Display Ciphertext File	User can view output before exiting.



Class Diagram:

Version 1:

